Bases de données avancées : normalisation et transactions

Informatique — Licence 2 (L2)

Cours INF-L2-05

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Concevoir des bases saines et fiables : détecter les anomalies, raisonner en dépendances fonctionnelles, normaliser jusqu'à la BCNF, puis protéger les données par les transactions ACID, le contrôle de concurrence, les droits d'accès, les déclencheurs et les procédures stockées.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Anomalies de mise à jour et dépendances fonctionnelles
• Leçon 2 — Les formes normales : de la première forme à la BCNF
• Leçon 3 — Transactions et propriétés ACID
• Leçon 4 — Contrôle de concurrence et verrouillage
• Leçon 5 — Gestion des utilisateurs et des droits d'accès
• Leçon 6 — Déclencheurs et procédures stockées : initiation
• Leçon 7 — Étude de cas : refondre une base de données mal conçue
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à créer des tables et à les interroger en SQL. Mais créer une table qui « fonctionne » ne suffit pas : une table mal pensée accepte les données, puis les contredit. Le nom d'un client est répété à chaque commande, une adresse corrigée à un endroit reste fausse à un autre, et supprimer une vieille commande fait disparaître un produit du catalogue. Ce cours t'apprend à concevoir des schémas qui empêchent ces incohérences par construction : c'est la normalisation, fondée sur une notion simple et puissante, la dépendance fonctionnelle.

La deuxième moitié du cours protège les données pendant leur vie quotidienne. Quand deux caissiers vendent le même article au même instant, quand un virement est interrompu par une coupure de courant entre le débit et le crédit, la base doit rester juste. Les transactions et leurs propriétés ACID, les verrous, les niveaux d'isolation et la gestion fine des droits d'accès sont les outils que le SGBD met à ta disposition pour cela.

Le cours se termine par les déclencheurs et les procédures stockées, qui déplacent une partie des règles métier dans la base elle-même, puis par une étude de cas complète : nous reprendrons une table unique, pleine d'anomalies, et nous la transformerons pas à pas en un schéma propre, testé et requêtable. Les exemples SQL exécutables ont été vérifiés sous SQLite ; ceux qui relèvent de MySQL (droits, procédures) sont signalés explicitement.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Repérer les trois anomalies (insertion, modification, suppression) dans une table mal conçue.
- Écrire les dépendances fonctionnelles d'une relation à partir des règles du métier et en déduire ses clés candidates.
- Décomposer une relation, sans perte d'information, jusqu'en troisième forme normale puis en forme normale de Boyce-Codd.
- Expliquer les quatre propriétés ACID et conduire une transaction avec BEGIN, COMMIT, ROLLBACK et SAVEPOINT.
- Identifier les problèmes de concurrence (perte de mise à jour, lecture impropre, lecture non reproductible, lignes fantômes) et choisir le niveau d'isolation adapté.
- Attribuer des droits d'accès selon le principe du moindre privilège (GRANT, REVOKE, rôles).
- Créer un déclencheur d'audit ou de contrôle et une procédure stockée simple, en sachant quel SGBD les prend en charge.

Leçon 1 — Anomalies de mise à jour et dépendances fonctionnelles

Imagine le centre de formation d'une ONG à Bamako qui gère ses inscriptions dans une seule table INSCRIPTION : chaque ligne mélange l'étudiant (nom, téléphone), le module (nom du module, enseignant, département de l'enseignant, prix) et l'inscription elle-même (date). Cette table « contient tout », et c'est précisément son problème : une même information y est écrite de nombreuses fois. Le département de M. Koné, par exemple, est répété sur chaque ligne de chaque étudiant inscrit à son module. Cette répétition inutile s'appelle la redondance, et elle est la mère de toutes les anomalies.

On distingue trois anomalies. L'anomalie de modification : si M. Koné change de département, il faut corriger toutes ses lignes ; en oublier une suffit à rendre la base incohérente, avec deux réponses possibles à la même question. L'anomalie d'insertion : impossible d'enregistrer un nouveau module tant qu'aucun étudiant ne s'y est inscrit, car la clé de la table exige un étudiant. L'anomalie de suppression : si la seule inscrite à un module se désinscrit, supprimer sa ligne fait disparaître le module lui-même, son prix et son enseignant. Une bonne conception doit rendre ces trois situations impossibles.

Pour raisonner proprement, on utilise les dépendances fonctionnelles (DF). On écrit X -> Y, et on lit « X détermine Y », lorsque deux lignes qui ont la même valeur de X ont forcément la même valeur de Y. Les DF ne se devinent pas sur les données du moment : elles traduisent les règles du métier. Ici : un numéro d'étudiant détermine son nom et son téléphone ; un code module détermine le nom du module, son enseignant et son prix ; un enseignant appartient à un seul département, donc nomEnseignant -> deptEnseignant ; et c'est le couple (numEtu, codeMod) qui détermine la date d'inscription. À partir des DF, on calcule la clé : c'est l'ensemble minimal d'attributs qui détermine tous les autres — ici (numEtu, codeMod). Toute la normalisation consiste ensuite à traquer les DF dont la source n'est pas une clé.

L'essentiel
Une table unique mélange plusieurs sujets et crée redondances puis anomalies d'insertion, de modification et de suppression. Les dépendances fonctionnelles X -> Y expriment les règles du métier ; la clé est l'ensemble minimal d'attributs qui détermine tous les autres.

Exemple — La table INSCRIPTION, ses dépendances fonctionnelles et ses anomalies

```
-- Table unique du centre de formation (extrait)
INSCRIPTION(numEtu, nomEtu, telEtu, codeMod, nomMod,
nomEnseignant, deptEnseignant, prixMod, dateInscr)

E12 | Awa Diallo | 76 00 11 22 | BDD | Bases de données
| M. Koné | Informatique | 60 000 | 2026-09-14
E12 | Awa Diallo | 76 00 11 22 | RES | Réseaux
| Mme Touré | Informatique | 55 000 | 2026-09-15
E15 | Moussa Sy | 66 55 44 33 | BDD | Bases de données
| M. Koné | Informatique | 60 000 | 2026-09-21

Dépendances fonctionnelles (règles du métier) :
numEtu -> nomEtu, telEtu
codeMod -> nomMod, nomEnseignant, prixMod
nomEnseignant -> deptEnseignant
(numEtu, codeMod) -> dateInscr
Clé de la table : (numEtu, codeMod)

Anomalies visibles :
- Modification : « Informatique » est répété pour M. Koné ;
s'il change de département, toutes ses lignes sont à corriger.
- Insertion : un nouveau module sans inscrit ne peut pas
être enregistré (la clé exige un numEtu).
- Suppression : si Awa, seule inscrite au module RES dans
cet extrait, se désinscrit, le module RES disparaît.
```

À retenir

- Redondance = la même information écrite plusieurs fois ; c'est elle qui rend les anomalies possibles.
- Une DF X -> Y se lit « X détermine Y » et traduit une règle du métier, pas un hasard des données présentes.
- La clé se déduit des DF : plus petit ensemble d'attributs qui détermine tous les autres.

Exercice

Une petite entreprise tient ses livraisons dans la table LIVRAISON(numFourn, nomFourn, villeFourn, codeProd, nomProd, qteLivree, dateLiv). Règles du métier : un fournisseur a un seul nom et une seule ville ; un produit a un seul nom ; la quantité livrée dépend du fournisseur, du produit ET de la date de livraison. 1) Écris toutes les dépendances fonctionnelles. 2) Donne la clé de la table. 3) Cite un exemple concret de chacune des trois anomalies.

✔ Corrigé
1) DF : numFourn -> nomFourn, villeFourn ; codeProd -> nomProd ; (numFourn, codeProd, dateLiv) -> qteLivree. 2) La clé est (numFourn, codeProd, dateLiv) : il faut les trois pour déterminer la quantité, et aucun sous-ensemble ne suffit. 3) Modification : si le fournisseur F3 déménage de Ségou à Sikasso, il faut corriger toutes ses lignes de livraison, sinon la base donne deux villes pour F3. Insertion : impossible d'enregistrer un nouveau produit (ou un nouveau fournisseur) tant qu'aucune livraison ne le concerne. Suppression : si l'on supprime la seule livraison d'un produit, son nom disparaît de la base alors que le produit existe toujours au catalogue.

Leçon 2 — Les formes normales : de la première forme à la BCNF

Normaliser, c'est décomposer une relation en plusieurs relations plus petites, sans perdre d'information, jusqu'à ce que chaque fait soit rangé à sa place. La première forme normale (1FN) pose le socle : chaque attribut doit contenir une valeur atomique, c'est-à-dire indivisible. Un champ téléphone contenant « 76 00 11 22 / 66 55 44 33 » viole la 1FN : impossible de rechercher proprement le second numéro. On corrige en ne gardant qu'un numéro par ligne, ou mieux, en créant une table TELEPHONE(numEtu, telephone) si un étudiant peut en avoir plusieurs. Sans clé ni valeurs atomiques, aucune autre forme normale n'a de sens.

La deuxième forme normale (2FN) s'attaque aux dépendances partielles : dans une relation en 1FN, aucun attribut non clé ne doit dépendre d'une partie seulement de la clé. Ce défaut n'existe que si la clé est composée. Dans INSCRIPTION, de clé (numEtu, codeMod), le nom de l'étudiant dépend de numEtu seul : c'est une dépendance partielle, source de redondance. On décompose alors en isolant chaque déterminant avec ce qu'il détermine : ETUDIANT(numEtu, nomEtu, telEtu), MODULE(codeMod, nomMod, nomEnseignant, deptEnseignant, prixMod) et INSCRIPTION(numEtu, codeMod, dateInscr). La décomposition est sans perte car la jointure des trois tables, sur les attributs communs qui sont des clés, reconstruit exactement la table de départ.

La troisième forme normale (3FN) élimine les dépendances transitives : un attribut non clé ne doit pas dépendre d'un autre attribut non clé. Dans MODULE, codeMod -> nomEnseignant et nomEnseignant -> deptEnseignant : le département dépend de la clé par l'intermédiaire de l'enseignant. On extrait donc ENSEIGNANT(nomEnseignant, deptEnseignant). Enfin, la forme normale de Boyce-Codd (BCNF) va un pas plus loin : pour TOUTE dépendance non triviale X -> Y, X doit être une super-clé. Une relation peut être en 3FN sans être en BCNF lorsqu'un attribut qui détermine fait partie d'une clé candidate sans être lui-même une clé. La BCNF est atteignable sans perte, mais parfois au prix de la préservation de certaines DF, qu'il faudra alors contrôler autrement (contrainte ou déclencheur).

L'essentiel
1FN : valeurs atomiques. 2FN : pas de dépendance partielle sur une partie de clé composée. 3FN : pas de dépendance transitive via un attribut non clé. BCNF : tout déterminant d'une DF non triviale est une super-clé. Chaque décomposition doit être sans perte d'information.

Exemple — Décomposition pas à pas d'INSCRIPTION jusqu'en 3FN, puis le cas BCNF

```
Point de départ (après mise en 1FN : téléphones atomiques)
INSCRIPTION(numEtu, nomEtu, telEtu, codeMod, nomMod,
nomEnseignant, deptEnseignant, prixMod, dateInscr)
Clé : (numEtu, codeMod)

Étape 2FN — chasser les dépendances PARTIELLES :
numEtu -> nomEtu, telEtu
codeMod -> nomMod, nomEnseignant, deptEnseignant, prixMod
ETUDIANT(numEtu, nomEtu, telEtu)
MODULE(codeMod, nomMod, nomEnseignant, deptEnseignant, prixMod)
INSCRIPTION(numEtu, codeMod, dateInscr)

Étape 3FN — dans MODULE, dépendance TRANSITIVE :
codeMod -> nomEnseignant -> deptEnseignant
MODULE(codeMod, nomMod, prixMod, nomEnseignant)
ENSEIGNANT(nomEnseignant, deptEnseignant)

Le cas BCNF — relation SUIVI(etudiant, matiere, enseignant)
Règles : un étudiant suit une matière avec UN enseignant ;
chaque enseignant n'enseigne qu'UNE matière.
(etudiant, matiere) -> enseignant
enseignant -> matiere
Clés candidates : (etudiant, matiere) et (etudiant, enseignant).
« enseignant » détermine « matiere » sans être une super-clé :
SUIVI est en 3FN (matiere appartient à une clé candidate)
mais PAS en BCNF. Décomposition sans perte :
ENSEIGNE(enseignant, matiere)
SUIVI2(etudiant, enseignant)
```

À retenir

- On ne peut tester la 2FN que si la clé est composée ; avec une clé simple, toute relation en 1FN est déjà en 2FN.
- Dépendance transitive = clé -> attribut non clé -> autre attribut non clé : on extrait le maillon du milieu dans sa propre table.
- BCNF : le déterminant de chaque DF non triviale doit être une super-clé ; la décomposition BCNF est sans perte mais ne préserve pas toujours toutes les DF.

Exercice

On considère COMMANDE2(numCmd, dateCmd, numClient, nomClient, adresseClient) avec les DF : numCmd -> dateCmd, numClient ; numClient -> nomClient, adresseClient. 1) Quelle est la clé ? 2) La relation est-elle en 2FN ? en 3FN ? Justifie. 3)

Décompose-la en 3FN et vérifie que chaque table obtenue est même en BCNF.

✔ Corrigé
1) La clé est numCmd : il détermine directement dateCmd et numClient, donc indirectement nomClient et adresseClient. 2) 2FN : oui, car la clé est simple (un seul attribut) — aucune dépendance partielle n'est possible. 3FN : non, car nomClient et adresseClient dépendent de la clé par transitivité, via numClient qui n'est pas une clé de COMMANDE2. 3) Décomposition sans perte : COMMANDE(numCmd, dateCmd, numClient) et CLIENT(numClient, nomClient, adresseClient). Dans COMMANDE, le seul déterminant est numCmd, qui est la clé ; dans CLIENT, c'est numClient, également la clé : les deux tables sont donc en BCNF, et la jointure sur numClient (clé de CLIENT) reconstruit les données d'origine.

Leçon 3 — Transactions et propriétés ACID

Une transaction est une suite d'opérations que le SGBD traite comme un tout indivisible : soit toutes réussissent et leurs effets deviennent définitifs (COMMIT, la validation), soit l'une d'elles échoue et tout est annulé (ROLLBACK), comme si rien ne s'était passé. L'exemple canonique est le virement : débiter le compte C1 de 50 000 F puis créditer le compte C2. Si le système tombe en panne entre les deux opérations, l'argent aurait disparu. Encadrer les deux UPDATE par BEGIN et COMMIT garantit qu'ils seront validés ensemble — ou pas du tout. En SQL, on ouvre la transaction par BEGIN (START TRANSACTION sous MySQL), on la termine par COMMIT ou ROLLBACK, et SAVEPOINT permet de poser des jalons pour annuler seulement une partie du travail.

L'atomicité est la propriété « tout ou rien » que l'on vient de décrire : elle est assurée par le journal de transactions, dans lequel le SGBD note les opérations pour pouvoir les défaire. La cohérence signifie que la transaction fait passer la base d'un état valide à un autre état valide : toutes les contraintes (clés, CHECK, clés étrangères, règles exprimées) sont respectées à la fin. Dans l'exemple du virement, la contrainte CHECK (solde >= 0) refuse un débit supérieur au solde ; la transaction avorte et la cohérence est préservée. Attention : la cohérence dépend aussi de la justesse des contraintes déclarées — le SGBD ne connaît que les règles qu'on lui a données.

L'isolation garantit que des transactions exécutées en même temps ne se voient pas dans un état intermédiaire : chacune travaille comme si elle était seule, et les effets d'une transaction non encore validée restent invisibles pour les autres. La durabilité, enfin, promet qu'une fois le COMMIT acquitté, les données survivent aux pannes, même à une coupure de courant : le SGBD écrit d'abord les modifications dans un journal stocké sur disque (journalisation, souvent appelée WAL — Write Ahead Logging) avant de confirmer la validation. Une transaction traverse ainsi plusieurs états : active, partiellement validée, validée, échouée puis abandonnée après son ROLLBACK. Retiens l'acronyme : Atomicité, Cohérence, Isolation, Durabilité — ACID.

L'essentiel
Une transaction regroupe des opérations validées ensemble par COMMIT ou annulées ensemble par ROLLBACK. ACID : Atomicité (tout ou rien), Cohérence (d'un état valide à un autre), Isolation (pas d'états intermédiaires visibles), Durabilité (un COMMIT survit aux pannes, grâce au journal).

Exemple — Un virement bancaire testé sous SQLite : COMMIT, puis ROLLBACK sur contrainte violée

```
-- Table des comptes (testée sous SQLite)
CREATE TABLE compte (
numero TEXT PRIMARY KEY,
titulaire TEXT,
solde INTEGER CHECK (solde >= 0) -- jamais de solde négatif
);
INSERT INTO compte VALUES ('C1', 'Awa Diallo', 120000);
INSERT INTO compte VALUES ('C2', 'Moussa Traoré', 30000);

-- Virement de 50 000 F de C1 vers C2 : tout ou rien
BEGIN;
UPDATE compte SET solde = solde - 50000 WHERE numero = 'C1';
UPDATE compte SET solde = solde + 50000 WHERE numero = 'C2';
COMMIT;
-- Résultat vérifié : C1 = 70 000 ; C2 = 80 000

-- Même virement tenté avec 200 000 F : le CHECK bloque
BEGIN;
UPDATE compte SET solde = solde - 200000 WHERE numero = 'C1';
-- ERREUR : contrainte CHECK (solde >= 0) violée
ROLLBACK;
-- Résultat vérifié : les soldes restent 70 000 et 80 000.
-- Sans transaction, le débit aurait pu rester écrit seul :
-- l'argent se serait évaporé.
```

À retenir

- BEGIN ouvre, COMMIT valide définitivement, ROLLBACK annule tout depuis le BEGIN ; SAVEPOINT crée un point de retour partiel.
- L'atomicité repose sur le journal ; la durabilité repose sur l'écriture du journal sur disque avant l'acquittement du COMMIT.
- Une contrainte violée (CHECK, clé étrangère…) fait échouer l'opération : à l'application de prévoir le ROLLBACK.

Exercice

En reprenant la table compte de l'exemple (soldes : C1 = 70 000, C2 = 80 000 après le premier virement), on exécute le script suivant : BEGIN ; UPDATE compte SET solde = solde - 10000 WHERE numero = 'C1' ; SAVEPOINT apres_debit ; UPDATE compte SET solde = solde + 10000 WHERE numero = 'C2' ; ROLLBACK TO apres_debit ; COMMIT ; Quels sont les soldes finaux de C1 et C2 ? Explique précisément ce que le ROLLBACK TO a annulé et ce qu'il a conservé.

✔ Corrigé
Soldes finaux vérifiés : C1 = 60 000 et C2 = 80 000. Le débit de 10 000 sur C1 a bien eu lieu AVANT le point de sauvegarde apres_debit : il est conservé et validé par le COMMIT final. Le crédit de 10 000 sur C2, effectué APRÈS le SAVEPOINT, est annulé par le ROLLBACK TO apres_debit, qui ramène la transaction à l'état du jalon sans l'abandonner entièrement. Résultat : l'argent a quitté C1 sans arriver sur C2 — ce script serait une erreur métier pour un virement, mais il illustre parfaitement le rôle d'un SAVEPOINT : annuler une partie récente du travail (ici le crédit) tout en gardant le reste de la transaction en vie.

Leçon 4 — Contrôle de concurrence et verrouillage

Une base réelle sert des dizaines d'utilisateurs en même temps : pendant qu'un vendeur encaisse, un autre consulte le stock et le gérant affiche le chiffre d'affaires. Sans précaution, ces accès simultanés corrompent les données. Le pire cas est la perte de mise à jour : deux vendeurs lisent le même stock de 10 articles ; le premier vend 1 article et écrit 9 ; le second, qui avait aussi lu 10, vend 2 articles et écrit 8. Trois articles sont partis, mais le stock affiche 8 au lieu de 7 : la mise à jour du premier a été écrasée par celle du second, qui travaillait sur une valeur périmée. Aucune des deux transactions n'a « mal » fonctionné prise isolément : c'est leur entrelacement qui est fautif.

On classe les lectures problématiques en trois phénomènes. La lecture impropre (dirty read) : une transaction lit une donnée modifiée par une autre transaction non encore validée — si celle-ci fait un ROLLBACK, la première a lu une valeur qui n'a jamais existé. La lecture non reproductible : une transaction relit la même ligne deux fois et obtient deux valeurs différentes, car une autre transaction l'a modifiée et validée entre-temps. Les lignes fantômes : la même requête, relancée, renvoie des lignes supplémentaires insérées par une autre transaction validée entre les deux lectures. Le niveau d'isolation choisi règle lesquels de ces phénomènes sont tolérés : READ UNCOMMITTED les tolère tous, READ COMMITTED (souvent le défaut) interdit les lectures impropres, REPEATABLE READ ajoute la stabilité des lignes relues, et SERIALIZABLE interdit tout, au prix de performances moindres.

Le mécanisme classique derrière l'isolation est le verrouillage. Avant de lire une donnée, une transaction pose un verrou partagé (plusieurs lecteurs peuvent coexister) ; avant d'écrire, elle exige un verrou exclusif, incompatible avec tous les autres. Le verrouillage à deux phases impose de prendre tous ses verrous avant d'en relâcher aucun, ce qui garantit des exécutions équivalentes à un ordre séquentiel. La granularité compte : verrouiller une ligne bloque peu de monde, verrouiller toute la table bloque tout le monde mais coûte moins cher à gérer (SQLite, par exemple, verrouille la base entière en écriture). Reste le risque d'interblocage (deadlock) : T1 détient la ligne A et attend B, T2 détient B et attend A. Le SGBD le détecte et sacrifie l'une des deux transactions par un ROLLBACK, quitte à la relancer ensuite.

L'essentiel
Trois phénomènes à connaître : perte de mise à jour, lecture impropre, lecture non reproductible et lignes fantômes. Le niveau d'isolation (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE) choisit le compromis entre sécurité et performance ; il est mis en œuvre par des verrous partagés et exclusifs, avec risque d'interblocage à gérer.

Exemple — Perte de mise à jour déroulée dans le temps, et carte des niveaux d'isolation

```
-- Perte de mise à jour : stock du produit P = 10
Temps Vendeur A Vendeur B
t1 lit stock : 10
t2 lit stock : 10
t3 vend 1 -> écrit 9
t4 vend 2 -> écrit 8
=> 3 articles vendus, mais le stock affiche 8 au lieu de 7 :
la mise à jour de A a été PERDUE, écrasée par celle de B.
Avec un verrou exclusif pris dès la lecture « pour
modification », B aurait attendu la validation de A,
puis lu 9 et écrit 7.

-- Niveaux d'isolation : phénomènes encore possibles
Niveau Lecture Lecture non Lignes
impropre reproductible fantômes
READ UNCOMMITTED possible possible possible
READ COMMITTED évitée possible possible
REPEATABLE READ évitée évitée possible (*)
SERIALIZABLE évitée évitée évitée

(*) En pratique, MySQL/InnoDB empêche aussi les fantômes
en REPEATABLE READ grâce à ses verrous d'intervalle.
SQLite, lui, fonctionne en pratique en mode sérialisé
pour les écritures : un seul rédacteur à la fois.
```

À retenir

- Lecture impropre = lire une donnée qu'un ROLLBACK peut faire disparaître ; READ COMMITTED l'interdit.
- Verrou partagé : lecture concurrente autorisée ; verrou exclusif : un seul détenteur, exigé pour écrire.
- Interblocage (deadlock) : deux transactions s'attendent mutuellement ; le SGBD en annule une, l'application doit savoir la relancer.

Exercice

Dans un service de billetterie, la transaction T1 (rapport du matin) lit le nombre de places vendues, puis, quelques secondes plus tard, relit le même nombre pour vérifier un total. Entre les deux lectures, la transaction T2 vend 20 places et valide. T1 obtient deux valeurs différentes. 1) Comment s'appelle ce phénomène ? 2) Quel niveau d'isolation minimal faut-il pour l'empêcher ? 3) Si T1 avait lu une valeur modifiée par

T2 AVANT la validation de T2, de quel phénomène plus grave s'agirait-il, et quel niveau suffit à l'éviter ?

✔ Corrigé
1) C'est une lecture non reproductible : la même ligne, relue par la même transaction, a changé entre les deux lectures à cause d'une transaction concurrente validée. 2) Il faut REPEATABLE READ (ou SERIALIZABLE) : la transaction conserve alors une vision stable des lignes qu'elle a lues, typiquement grâce aux verrous de lecture conservés ou à un instantané. READ COMMITTED ne suffit pas, car il autorise ce phénomène. 3) Il s'agirait d'une lecture impropre (dirty read) : T1 aurait utilisé une donnée « provisoire » que le ROLLBACK éventuel de T2 aurait rendue fictive. READ COMMITTED suffit à l'éviter, car on n'y lit que des données validées.

Leçon 5 — Gestion des utilisateurs et des droits d'accès

Toutes les protections du monde ne servent à rien si tout le monde se connecte avec le même compte tout-puissant. La gestion des droits répond à une question simple : qui a le droit de faire quoi, sur quelles données ? Le principe directeur est le moindre privilège : chaque compte ne reçoit que les droits strictement nécessaires à sa tâche, ni plus, ni sur plus de tables que nécessaire. Un caissier doit pouvoir lire les produits et insérer des commandes, mais jamais modifier la table des salaires ni supprimer l'historique des ventes. Si son mot de passe fuit, les dégâts restent limités à ses seuls droits — c'est toute la différence entre un incident et une catastrophe.

En MySQL, on crée un compte par CREATE USER, on distribue les droits par GRANT et on les retire par REVOKE ; SHOW GRANTS affiche les droits effectifs d'un compte. Les privilèges se donnent à plusieurs échelles : global (toutes les bases), base, table, et même colonne — on peut autoriser la mise à jour de la seule colonne stock d'une table produit. PostgreSQL utilise une syntaxe très proche. SQLite, en revanche, est une base « embarquée » sans comptes : la protection repose alors sur les droits du fichier dans le système d'exploitation, ce qu'il faut savoir avant de lui confier des données sensibles. Pour ne pas exposer les données complètes, on combine souvent les droits avec des vues : donner SELECT sur une vue qui masque les colonnes sensibles vaut mieux que donner accès à la table entière.

Depuis MySQL 8, les rôles simplifient l'administration : on crée un rôle (par exemple role_caisse) auquel on attribue un paquet de droits, puis on attribue le rôle aux comptes des caissiers. Quand les missions évoluent, on ajuste le rôle une seule fois au lieu de repasser sur chaque compte. Bonnes pratiques à retenir : un compte distinct par personne et par application (jamais le compte root pour l'application du quotidien), des mots de passe robustes gérés par le SGBD sous forme hachée, la révocation immédiate des droits des personnes qui partent, et un audit régulier — qui possède encore des droits qu'il n'utilise plus ? La sécurité d'une base est d'abord une affaire d'organisation.

L'essentiel
Principe du moindre privilège : chaque compte reçoit uniquement les droits utiles à sa mission. CREATE USER crée un compte, GRANT attribue, REVOKE retire, SHOW GRANTS contrôle ; les rôles regroupent des droits réutilisables. SQLite n'a pas de comptes : sa sécurité dépend des droits sur le fichier.

Exemple — Créer des comptes et distribuer les droits, en syntaxe MySQL

```
-- Syntaxe MySQL (SQLite ne gère pas les comptes)

-- 1. Un compte par métier, jamais de compte partagé
CREATE USER 'caissier'@'localhost' IDENTIFIED BY '...';
CREATE USER 'consultation'@'%' IDENTIFIED BY '...';

-- 2. Le strict nécessaire (moindre privilège)
GRANT SELECT, INSERT ON boutique.commande
TO 'caissier'@'localhost';
GRANT UPDATE (stock) ON boutique.produit
TO 'caissier'@'localhost';
GRANT SELECT ON boutique.vue_ventes_publiques
TO 'consultation'@'%';
-- 'consultation' passe par une vue : les colonnes
-- sensibles (marges, salaires) restent invisibles.

-- 3. Retirer un droit devenu inutile
REVOKE INSERT ON boutique.commande FROM 'caissier'@'localhost';

-- 4. Contrôler les droits effectifs
SHOW GRANTS FOR 'caissier'@'localhost';

-- 5. MySQL 8 : regrouper les droits dans un rôle
CREATE ROLE 'role_caisse';
GRANT SELECT ON boutique.produit TO 'role_caisse';
GRANT SELECT, INSERT ON boutique.commande TO 'role_caisse';
GRANT 'role_caisse' TO 'caissier'@'localhost';
```

À retenir

- Moindre privilège : ni droits superflus, ni tables superflues, ni durée superflue — tout droit inutile est un risque.
- GRANT / REVOKE / SHOW GRANTS sont le trio de base ; les privilèges peuvent descendre jusqu'à la colonne.
- Une vue + un droit de lecture sur la vue protège les colonnes sensibles sans dupliquer les données.

Exercice

La boutique veut un compte pour un stagiaire qui saisit les commandes pendant deux mois. Écris les ordres MySQL pour : 1) créer le compte 'stagiaire'@'localhost' ; 2) lui donner SELECT sur produit, et SELECT + INSERT sur commande et ligne_commande, mais aucun droit de modification ni de suppression ; 3) à la fin du stage, retirer le droit

d'insertion sur commande tout en gardant ses lectures pour le bilan. 4) Explique pourquoi on ne lui donne pas simplement tous les droits « le temps du stage ».

✔ Corrigé
1) CREATE USER 'stagiaire'@'localhost' IDENTIFIED BY '...' ; 2) GRANT SELECT ON boutique.produit TO 'stagiaire'@'localhost' ; GRANT SELECT, INSERT ON boutique.commande TO 'stagiaire'@'localhost' ; GRANT SELECT, INSERT ON boutique.ligne_commande TO 'stagiaire'@'localhost' ; 3) REVOKE INSERT ON boutique.commande FROM 'stagiaire'@'localhost' ; (on révoquerait de même l'INSERT sur ligne_commande si le bilan ne demande que des lectures ; à terme, le compte doit être supprimé par DROP USER). 4) « Tous les droits le temps du stage » viole le moindre privilège : avec UPDATE et DELETE, une erreur de manipulation ou un mot de passe compromis peut effacer ou falsifier l'historique des ventes, et le droit d'accès aux tables sensibles (salaires, utilisateurs) n'a aucun lien avec sa mission. La période courte ne réduit pas le risque : la plupart des dégâts se font en quelques minutes.

Leçon 6 — Déclencheurs et procédures stockées : initiation

Un déclencheur (trigger) est une procédure que le SGBD exécute automatiquement quand un événement précis survient sur une table : INSERT, UPDATE ou DELETE, juste avant (BEFORE) ou juste après (AFTER) l'opération, ligne par ligne. Le déclencheur accède à l'ancienne valeur (OLD) et à la nouvelle (NEW). Ses trois usages classiques : tenir un journal d'audit (qui a changé quoi, quand, de quelle valeur à quelle valeur), faire respecter une règle qu'une simple contrainte ne peut pas exprimer (par exemple interdire un stock négatif en bloquant la mise à jour), et maintenir des données dérivées (compteurs, totaux). Comme il vit dans la base, il protège les données quel que soit le programme qui écrit : application web, script ou requête manuelle.

La procédure stockée est différente : elle n'est pas automatique, on l'appelle explicitement (CALL) avec des paramètres d'entrée (IN), de sortie (OUT) ou les deux. Elle regroupe, côté serveur, une suite d'instructions SQL et de logique (conditions, boucles, gestion d'erreur), souvent une transaction complète. Avantages : le code métier est écrit une fois, partagé par toutes les applications, et l'aller-retour réseau se réduit à un seul appel. Inconvénients : la syntaxe varie d'un SGBD à l'autre (le langage de MySQL n'est pas celui de PostgreSQL, et SQLite ne connaît pas les procédures stockées), ce qui nuit à la portabilité ; la logique devient aussi moins visible pour les développeurs applicatifs. Déclencheurs et procédures existent dans MySQL et PostgreSQL ; SQLite connaît les déclencheurs mais pas les procédures stockées.

Il faut donc choisir où placer chaque règle. Les contraintes (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK) restent l'outil de base, déclaratif et toujours actif. Le déclencheur convient aux règles transversales liées aux données elles-mêmes — audit, garde-fou, recalcul — mais il faut le documenter soigneusement, car une action « invisible » qui modifie d'autres tables surprend toujours quelqu'un un jour. La procédure stockée convient aux opérations métier complètes et fréquentes (valider une commande, clôturer un mois). Dans l'exemple qui suit, les deux déclencheurs sont écrits et testés sous SQLite ; la procédure, elle, est en syntaxe MySQL, et les deux

mécanismes se complètent : l'appel de la procédure de réassort provoque une mise à jour du stock, que le déclencheur d'audit enregistre automatiquement.

L'essentiel
Déclencheur : code exécuté automatiquement par le SGBD sur INSERT/UPDATE/DELETE, BEFORE ou AFTER, avec accès à OLD et NEW — parfait pour l'audit et les garde-fous. Procédure stockée : code SQL appelé explicitement (CALL) avec paramètres IN/OUT, idéal pour encapsuler une opération métier ; attention, syntaxe propre à chaque SGBD et absente de SQLite.

Exemple — Audit de stock et garde anti-stock négatif (SQLite), réassort par procédure (MySQL)

```
-- Déclencheurs TESTÉS sous SQLite
-- produit(code, nom, stock) ; journal_stock(code_produit,
-- ancien_stock, nouveau_stock, moment)
CREATE TRIGGER trg_stock_audit
AFTER UPDATE OF stock ON produit
BEGIN
INSERT INTO journal_stock(code_produit, ancien_stock,
nouveau_stock, moment)
VALUES (OLD.code, OLD.stock, NEW.stock, datetime('now'));
END;

CREATE TRIGGER trg_stock_positif
BEFORE UPDATE OF stock ON produit
WHEN NEW.stock < 0
BEGIN
SELECT RAISE(ABORT, 'Stock négatif interdit');
END;
-- Vérifié : passer un stock de 40 à 37 écrit une ligne de
-- journal (P1, 40, 37) ; tenter de le passer à -35 est
-- REFUSÉ par le déclencheur, le stock ne bouge pas.

-- Procédure stockée : syntaxe MySQL (absente de SQLite)
DELIMITER //
CREATE PROCEDURE Reassortir(IN p_code TEXT, IN p_qte INT)
BEGIN
UPDATE produit SET stock = stock + p_qte
WHERE code = p_code;
END //
DELIMITER ;
-- CALL Reassortir('P1', 20); -> le stock augmente de 20
-- et le déclencheur d'audit, lui, journalise la hausse.
```

À retenir

• Un trigger se déclenche sur événement (INSERT/UPDATE/DELETE), BEFORE ou AFTER ; OLD = avant, NEW = après.

• SQLite sait faire des triggers, dont le garde-fou RAISE(ABORT, ...) dans une clause WHEN, mais pas les procédures stockées.

• Règle d'or : contrainte d'abord, déclencheur pour ce que la contrainte ne sait pas dire, procédure pour les opérations métier complètes.

Exercice

La boutique tient ses ventes dans vente(code, qte) et son stock dans produit(code, nom, stock). Écris, en syntaxe SQLite, le déclencheur qui, à chaque insertion d'une vente, diminue automatiquement le stock du produit vendu de la quantité vendue. Précise le moment (BEFORE ou AFTER), l'événement, et les valeurs utilisées. Que se passe-t-il si la vente rend le stock négatif, en supposant que le déclencheur trg_stock_positif de l'exemple existe toujours ?

✔ Corrigé
CREATE TRIGGER trg_vente_stock AFTER INSERT ON vente BEGIN UPDATE produit SET stock = stock - NEW.qte WHERE code = NEW.code ; END ; Moment : AFTER (la vente est enregistrée, on en tire la conséquence sur le stock) ; événement : INSERT sur vente ; valeurs : NEW.qte et NEW.code désignent la ligne de vente qui vient d'être insérée. Si la vente rend le stock négatif, la mise à jour du stock déclenche à son tour trg_stock_positif (BEFORE UPDATE sur produit avec WHEN NEW.stock < 0), qui exécute RAISE(ABORT) : l'erreur remonte et, si l'insertion de la vente et la mise à jour sont dans la même transaction, tout est annulé — la vente fictive ne reste pas enregistrée. C'est la composition des deux garde-fous qui rend le système robuste.

Leçon 7 — Étude de cas : refondre une base de données mal conçue

La boutique « Chez Mariam », à Bamako, gère ses commandes dans une seule table COMMANDE_BRUTE : numéro et date de commande, identité complète du client (nom, téléphone, code et nom de sa ville), et la liste des produits achetés avec quantités, le tout sur la même ligne. Les symptômes sont ceux de la leçon 1 : le nom et le téléphone d'une cliente fidèle sont répétés sur chacune de ses commandes ; quand elle change de numéro, l'ancienne valeur survit sur les vieilles commandes ; impossible d'enregistrer un produit nouveau tant qu'il n'a pas été commandé ; et si l'on purge les vieilles commandes, on perd du même coup les clients qui n'ont commandé qu'une fois. Ajoute que la liste de produits dans une cellule viole la 1FN : impossible de calculer proprement le total d'une commande.

On applique la méthode. D'abord les DF, lues sur les règles du métier : numCmd -> dateCmd, idClient ; idClient -> nomClient, telClient, codeVille ; codeVille -> nomVille ; codeProd -> nomProd, prixProd ; (numCmd, codeProd) -> qte. Mise en 1FN : on éclate la liste de produits en une ligne par produit commandé ; la clé devient (numCmd, codeProd). Passage en 2FN : les dépendances partielles sortent — numCmd seul détermine date et client, codeProd seul détermine nom et prix du produit — d'où COMMANDE(numCmd, dateCmd, idClient), PRODUIT(codeProd, nomProd, prixProd) et LIGNE_COMMANDE(numCmd, codeProd, qte). Passage en 3FN : dans les informations client, codeVille -> nomVille est une dépendance transitive depuis idClient ; on extrait VILLE(codeVille, nomVille) et CLIENT(idClient, nomClient, telClient, codeVille). Chaque déterminant est désormais une clé : le schéma final est en BCNF.

Vérifions que la refonte n'a rien perdu : le schéma final a été créé sous SQLite, alimenté avec trois commandes réelles, et la jointure des cinq tables reconstruit exactement les faits d'origine, ligne par ligne — c'est la preuve d'une décomposition sans perte. Les totaux calculés par jointure (39 500 F, 17 500 F, 18 000 F) servent de test de non-régression. On peut alors ajouter les renforts vus dans le cours : une transaction pour enregistrer commande et lignes d'un seul bloc, le déclencheur d'audit sur les stocks, et des droits différenciés (la caisse insère, le gérant consulte, personne ne supprime l'historique). Moralité : normaliser ne rend pas la base plus lente ou plus compliquée à interroger — les jointures sur clés indexées sont le quotidien du relationnel ; en revanche, ne pas normaliser rend chaque correction dangereuse.

L'essentiel
Méthode complète : relever les anomalies, écrire les DF, trouver la clé, décomposer 1FN -> 2FN -> 3FN/BCNF en suivant les déterminants, puis prouver la décomposition sans perte par une jointure de reconstruction et des totaux de contrôle, avant d'ajouter transactions, déclencheurs et droits.

Exemple — Le schéma refondu en 3FN/BCNF, créé et vérifié sous SQLite

```
-- Schéma final (3FN, en fait BCNF) — testé sous SQLite
CREATE TABLE ville (code_ville TEXT PRIMARY KEY,
nom_ville TEXT);
CREATE TABLE client (id_client INTEGER PRIMARY KEY,
nom TEXT, tel TEXT,
code_ville TEXT REFERENCES ville(code_ville));
CREATE TABLE produit (code_prod TEXT PRIMARY KEY,
nom_prod TEXT, prix INTEGER);
CREATE TABLE commande (num_cmd INTEGER PRIMARY KEY,
date_cmd TEXT,
id_client INTEGER REFERENCES client(id_client));
CREATE TABLE ligne_commande (
num_cmd INTEGER REFERENCES commande(num_cmd),
code_prod TEXT REFERENCES produit(code_prod),
qte INTEGER, PRIMARY KEY (num_cmd, code_prod));

-- Preuve « sans perte » : la jointure des cinq tables
-- reconstruit les faits d'origine. Sur le jeu d'essai
-- (3 commandes, 4 lignes), totaux obtenus par jointure :
-- commande 100 : Riz x2 + Huile x1 = 39 500 F
-- commande 101 : Riz x1 = 17 500 F
-- commande 102 : Huile x4 = 18 000 F
-- Aucune ligne perdue, aucun total modifié : la refonte
-- conserve l'information, les anomalies en moins.
```

À retenir

• On normalise à partir des dépendances fonctionnelles, jamais « au feeling » : chaque décomposition cite la DF qui la motive.

• Une décomposition doit être prouvée sans perte : jointure de reconstruction + quelques totaux de contrôle calculés des deux côtés.

• Après la structure, la fiabilité : transactions pour les écritures liées, déclencheurs pour l'audit, droits pour les personnes.

Exercice

Un club informatique note ses membres dans GESTION_NOTES(numEtu, nomEtu, filiereEtu, codeMatiere, nomMatiere, coefficient, note). Règles : un étudiant a un nom et une filière ; une matière a un nom et un coefficient ; la note dépend de l'étudiant ET de la matière. 1) Écris les DF et la clé. 2) Donne un exemple d'anomalie de suppression. 3) Décompose jusqu'en 3FN.

✔ Corrigé

1) DF : numEtu -> nomEtu, filiereEtu ; codeMatiere -> nomMatiere, coefficient ; (numEtu, codeMatiere) -> note. Clé : (numEtu, codeMatiere). 2) Suppression : si un étudiant est le seul à avoir suivi une matière optionnelle et qu'on supprime sa ligne (désinscription), la matière, son nom et son coefficient disparaissent du système alors qu'elle reste au programme. (Insertion et modification sont également possibles : impossible de créer une matière sans étudiant noté ; changer le coefficient impose de corriger toutes les lignes.) 3) Les dépendances partielles (numEtu détermine le nom, codeMatiere détermine le coefficient) violent la 2FN. Décomposition : ETUDIANT(numEtu, nomEtu, filiereEtu), MATIERE(codeMatiere, nomMatiere, coefficient), NOTE(numEtu, codeMatiere, note). Aucune dépendance transitive ne subsiste (il n'y a pas de DF entre attributs non clés) : le schéma est en 3FN, et même en BCNF puisque chaque déterminant y est une clé.

Travaux dirigés (TD)

TD 1 — Normaliser la table des emprunts d'une bibliothèque

La bibliothèque du campus gère ses prêts dans une table unique EMPRUNT(numLecteur, nomLecteur, numOuvrage, titreOuvrage, codeEditeur, nomEditeur, villeEditeur, dateEmprunt, dateRetour). Règles du métier : un lecteur a un seul nom ; un ouvrage a un seul titre et un seul éditeur ; un éditeur a un seul nom et une seule ville ; un emprunt est identifié par le couple (numLecteur, numOuvrage) car un lecteur ne peut pas emprunter deux fois le même ouvrage en même temps ; la date d'emprunt et la date de retour dépendent de ce couple. Travail : 1) Écris toutes les dépendances fonctionnelles. 2) Donne la clé de la table. 3) Explique une anomalie d'insertion et une anomalie de modification. 4) Décompose la relation étape par étape (2FN puis 3FN) en nommant la DF qui motive chaque décomposition. 5) Écris les CREATE TABLE du schéma final.

✔ Corrigé du TD 1
1) DF : numLecteur -> nomLecteur ; numOuvrage -> titreOuvrage, codeEditeur ; codeEditeur -> nomEditeur, villeEditeur ; (numLecteur, numOuvrage) -> dateEmprunt, dateRetour. 2) Clé : (numLecteur, numOuvrage). 3) Insertion : impossible d'enregistrer un nouvel ouvrage (ou un nouvel éditeur) tant qu'il n'a fait l'objet d'aucun prêt. Modification : si un éditeur déménage, sa ville doit être corrigée sur toutes les lignes de tous ses ouvrages, sous peine de réponses contradictoires. 4) Étape 2FN — dépendances partielles sur la clé composée : numLecteur seul détermine nomLecteur, numOuvrage seul détermine titreOuvrage et codeEditeur. On obtient LECTEUR(numLecteur, nomLecteur), OUVRAGE(numOuvrage, titreOuvrage, codeEditeur, nomEditeur, villeEditeur) et EMPRUNT(numLecteur, numOuvrage, dateEmprunt, dateRetour). Étape 3FN — dans OUVRAGE, codeEditeur -> nomEditeur, villeEditeur est une dépendance d'un attribut non clé vers d'autres attributs non clés (transitivité depuis numOuvrage) : on extrait EDITEUR(codeEditeur, nomEditeur, villeEditeur) et OUVRAGE devient (numOuvrage, titreOuvrage, codeEditeur). 5) Schéma final : CREATE TABLE lecteur(num_lecteur INTEGER PRIMARY KEY, nom TEXT) ; CREATE TABLE editeur(code_editeur TEXT PRIMARY KEY, nom TEXT, ville TEXT) ; CREATE TABLE ouvrage(num_ouvrage INTEGER PRIMARY KEY, titre TEXT, code_editeur TEXT REFERENCES editeur(code_editeur)) ; CREATE TABLE emprunt(num_lecteur INTEGER REFERENCES lecteur(num_lecteur), num_ouvrage INTEGER REFERENCES ouvrage(num_ouvrage), date_emprunt TEXT, date_retour TEXT, PRIMARY KEY (num_lecteur, num_ouvrage)). Chaque déterminant y est une clé : le schéma est en BCNF, et la jointure sur les clés étrangères reconstruit la table d'origine.

TD 2 — Concurrence à la caisse et transaction d'annulation

À la boutique « Chez Mariam », le stock du sac de riz affiche 7. Deux caisses, A et B, travaillent en même temps : à t1, A lit le stock (7) pour une vente de 2 sacs ; à t2, B lit le stock (7) pour une vente de 3 sacs ; à t3, A écrit 5 et valide ; à t4, B écrit 4 et valide. Par ailleurs, la gérante lance une consultation pendant une annulation en cours et obtient, pour la commande concernée, un montant déjà modifié alors que l'annulation sera finalement abandonnée (ROLLBACK). Travail : 1) Calcule le stock réel après les deux ventes et le stock affiché ; nomme le problème. 2) Nomme le phénomène dont la gérante est victime. 3) Pour chaque problème, indique un mécanisme qui l'aurait évité (verrouillage ou niveau d'isolation). 4) Écris, en SQL, la transaction d'annulation complète d'une commande qui avait fait sortir 4 bouteilles d'huile (produit HUI) : remettre le stock à niveau, supprimer les lignes de la commande 102 puis la commande elle-même, et explique pourquoi ces trois opérations doivent être atomiques.

✔ Corrigé du TD 2
1) Ventes réelles : 2 + 3 = 5 sacs, donc le stock réel devrait être 7 - 5 = 2. Le stock affiché est 4, la dernière écriture de B calculée depuis la valeur périmée 7 (7 - 3). C'est une perte de mise à jour : la vente de A a été ignorée par B. 2) La gérante a fait une lecture impropre (dirty read) : elle a lu une donnée modifiée par une transaction qui s'est terminée par un ROLLBACK, donc une valeur qui n'a jamais existé dans la base validée. 3) Perte de mise à jour : verrou exclusif sur la ligne de stock dès la lecture pour modification (ou niveau SERIALIZABLE) — B aurait attendu, lu 5, puis écrit 2. Lecture impropre : niveau READ COMMITTED au minimum, qui ne laisse lire que des données validées. 4) Transaction : BEGIN ; UPDATE produit SET stock = stock + 4 WHERE code_prod = 'HUI' ; DELETE FROM ligne_commande WHERE num_cmd = 102 ; DELETE FROM commande WHERE num_cmd = 102 ; COMMIT ; Atomicité indispensable : si le système s'arrête après la remise en stock mais avant les suppressions, la commande existerait encore avec un stock déjà restauré (huile comptée deux fois) ; si l'arrêt survient après les suppressions sans remise en stock, les 4 bouteilles disparaissent des comptes. BEGIN/COMMIT garantit les trois opérations ensemble, ou aucune.

Travaux pratiques (TP)

TP 1 — Refondre et sécuriser la base de la boutique, sous SQLite

Ouvre SQLite (par exemple via Python ou l'outil en ligne de commande sqlite3) et réalise la refonte complète. Étape 1 : crée le schéma normalisé de l'étude de cas (ville, client, produit, commande, ligne_commande) et insère le jeu d'essai : villes BKO/Bamako et SGU/Ségou ; clientes 1 « Aminata Keïta » (BKO) et 2 « Ibrahim Sangaré » (SGU) ; produits RIZ à 17 500 et HUI à 4 500 ; commandes 100 (cliente 1), 101 (client 2), 102 (cliente 1) ; lignes (100, RIZ, 2), (100, HUI, 1), (101, RIZ, 1), (102, HUI, 4). Étape 2 : écris la requête de reconstruction qui joint les cinq tables et calcule le montant de chaque ligne puis le total par commande. Étape 3 : ajoute une table produit_stock(code_prod, stock) initialisée à RIZ = 40 et HUI = 15, le déclencheur d'audit de la leçon 6 adapté, et le déclencheur garde-fou contre le stock négatif ; teste une baisse de stock licite puis une baisse impossible. Étape 4 : rejoue le virement de la leçon 3 entre deux comptes de la boutique (caisse et banque) avec un montant supérieur au solde, et prouve que le ROLLBACK a tout annulé.

✔ Attendus du TP 1
Résultats attendus, tous vérifiés lors de la préparation de ce cours. Étape 2 : la jointure renvoie 4 lignes ; totaux par commande : 100 -> 39 500 F (2 x 17 500 + 1 x 4 500), 101 -> 17 500 F, 102 -> 18 000 F (4 x 4 500). Si tes totaux diffèrent, cherche d'abord une erreur de saisie dans les lignes de commande avant de douter de la requête. Étape 3 : une mise à jour UPDATE produit_stock SET stock = stock - 3 pour RIZ fait passer le stock à 37 et écrit une ligne dans le journal (ancien 40, nouveau 37) ; une tentative de passer HUI de 15 à -35 (retrait de 50) est refusée par le déclencheur avec le message « Stock négatif interdit » et le stock reste à 15. Étape 4 : le virement valide de 50 000 donne (70 000, 80 000) ; le virement de 200 000 viole le CHECK (solde >= 0), le ROLLBACK laisse les soldes exactement à (70 000, 80 000). Ton TP est réussi si tu peux expliquer, pour chaque étape, QUEL mécanisme produit le résultat observé : jointure sur clés pour la reconstruction, OLD/NEW pour l'audit, WHEN + RAISE(ABORT) pour le garde-fou, journal de transactions pour le ROLLBACK.

Exercices corrigés supplémentaires

Exercice 1

Soit la relation R(A, B, C, D) et l'ensemble de DF F = { A -> B, B -> C, C -> D }. 1) Calcule la fermeture de A (tous les attributs que A détermine). 2) Quelle est la clé de R ? 3) R est-elle en 2FN ? en 3FN ? Justifie précisément. 4) Décompose R en relations BCNF, sans perte.

✔ Corrigé
1) Fermeture de A : A détermine B (par A -> B), B détermine C, C détermine D ; donc A+ = {A, B, C, D} : A détermine tout. 2) La clé est A, unique clé candidate, car aucun autre attribut ne détermine A et A détermine tous les attributs. 3) 2FN : oui — la clé est simple, aucune dépendance partielle n'est possible. 3FN : non — B -> C viole la 3FN (B n'est pas une super-clé et C n'appartient pas à une clé candidate), et C -> D également. 4) Décomposition en suivant les DF fautives : R1(A, B), R2(B, C), R3(C, D). Dans chaque relation, le seul déterminant est la clé (A dans R1, B dans R2, C dans R3) : les trois sont en BCNF. La décomposition est sans perte : R1 et R2 se joignent sur B, clé de R2 ; le résultat se joint à R3 sur C, clé de R3. Toutes les DF d'origine sont préservées, chacune vivant dans une des tables.

Exercice 2

On reprend la relation SUIVI(etudiant, matiere, enseignant) de la leçon 2, avec les DF : (etudiant, matiere) -> enseignant et enseignant -> matiere. 1) Liste les clés candidates. 2) SUIVI est-elle en 3FN ? en BCNF ? Justifie en citant la définition exacte utilisée dans chaque cas. 3) Propose une décomposition BCNF, prouve qu'elle est sans perte, et dis ce qu'elle ne préserve pas.

✔ Corrigé
1) Clés candidates : (etudiant, matiere) — elle détermine enseignant directement ; et (etudiant, enseignant) — elle détermine matiere par enseignant -> matiere, donc tout. 2) 3FN : oui. La seule DF gênante, enseignant -> matiere, a pour membre droit « matiere », qui appartient à une clé candidate : la définition de la 3FN tolère ce cas. BCNF : non — pour la BCNF, tout déterminant d'une DF non triviale doit être une super-clé ; or « enseignant » détermine « matiere » sans être une super-clé (il ne détermine pas « etudiant »). 3) Décomposition : ENSEIGNE(enseignant, matiere) et SUIVI2(etudiant, enseignant). Sans perte : l'attribut commun est « enseignant », qui est la clé de ENSEIGNE — le théorème de la jointure sans perte est satisfait. Ce qui n'est pas préservé : la DF (etudiant, matiere) -> enseignant ne peut plus être vérifiée dans une seule table ; pour garantir qu'un étudiant n'a qu'un enseignant par matière, il faudra une contrainte inter-tables ou un déclencheur. C'est le compromis classique de la BCNF.

Exercice 3

Écris la transaction SQL qui enregistre la commande 103 du client 2 achetant 3 sacs de riz (produit RIZ, prix 17 500, stock actuel géré dans produit_stock : 40) : insertion de la commande (date : 2026-10-06), insertion de la ligne (quantité 3), et décrémentation du stock. Calcule le montant de la commande. Puis dis ce qui se passe si un client commande 100 sacs alors que le déclencheur garde-fou trg_stock_positif (leçon 6) protège produit_stock.

✔ Corrigé
BEGIN ; INSERT INTO commande VALUES (103, '2026-10-06', 2) ; INSERT INTO ligne_commande VALUES (103, 'RIZ', 3) ; UPDATE produit_stock SET stock = stock - 3 WHERE code_prod = 'RIZ' ; COMMIT ; Montant de la commande : 3 x 17 500 = 52 500 F, stock de riz après coup : 37. Si la commande porte sur 100 sacs, la décrémentation ferait passer le stock à -60 : le déclencheur BEFORE UPDATE avec WHEN NEW.stock < 0 exécute RAISE(ABORT), l'erreur fait échouer la transaction entière et le ROLLBACK automatique annule aussi l'insertion de la commande et de sa ligne — il ne reste aucune trace d'une vente impossible. Sans la transaction, la commande fantôme resterait enregistrée : c'est exactement le genre d'incohérence que ce cours apprend à rendre impossible.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans une table de commandes où le nom du client est répété sur chaque commande, quel problème apparaît si le client change de nom et qu'une seule ligne est corrigée ?

A) Une anomalie d'insertion

B) Une anomalie de modification : la base devient incohérente

C) Une anomalie de suppression

2. L'écriture X -> Y signifie que…

A) X est toujours la clé primaire

B) X et Y ont le même type

C) Toute valeur de X est associée à une seule valeur de Y

3. Une relation est en première forme normale (1FN) si…

A) Tous ses attributs contiennent des valeurs atomiques

B) Elle ne contient aucune clé étrangère

C) Tous ses attributs sont des entiers

4. Une dépendance partielle ne peut exister que si…

A) La table contient beaucoup de lignes

B) La clé primaire est composée de plusieurs attributs

C) La table n'a pas de clé étrangère

5. La troisième forme normale interdit principalement…

A) Les clés composées

B) Les clés étrangères

C) Les dépendances transitives entre attributs non clés

6. Une relation est en BCNF si…

A) Tout déterminant d'une DF non triviale est une super-clé

B) Elle contient au plus trois tables

C) Toutes ses colonnes sont des clés

7. L'atomicité d'une transaction garantit que…

A) Les données sont chiffrées

B) Toutes ses opérations sont validées ensemble ou annulées ensemble

C) Elle s'exécute toujours en moins d'une seconde

8. Une lecture impropre (dirty read) se produit quand…

A) On lit une donnée modifiée par une transaction non encore validée

B) On lit lentement une table volumineuse

C) On lit une table sans index

9. Un verrou partagé posé par une transaction est compatible avec…

A) Un verrou exclusif d'une autre transaction

B) Un autre verrou partagé

C) Aucun autre verrou

10. Selon le principe du moindre privilège, l'ordre GRANT doit…

A) Donner tous les droits pour simplifier l'administration

B) Nommer précisément les opérations et les tables utiles au compte

C) Être remplacé par le partage du compte administrateur

Corrigés du QCM

1. Réponse B — Corriger une copie et pas les autres crée deux réponses contradictoires à la même question : c'est l'anomalie de modification, fille de la redondance.

2. Réponse C — Une dépendance fonctionnelle dit que le déterminant X fixe la valeur de Y : deux lignes égales sur X sont égales sur Y.

3. Réponse A — La 1FN exige l'atomicité : pas de liste ni d'ensemble de valeurs dans une cellule.

4. Réponse B — Dépendre « d'une partie de la clé » n'a de sens que si la clé a plusieurs parties : avec une clé simple, la 1FN entraîne déjà la 2FN.

5. Réponse C — En 3FN, un attribut non clé ne doit dépendre que de la clé, directement — jamais d'un autre attribut non clé.

6. Réponse A — La BCNF durcit la 3FN : même un déterminant dont le membre droit appartient à une clé candidate doit être lui-même une super-clé.

7. Réponse B — Tout ou rien : c'est le rôle du couple COMMIT / ROLLBACK et du journal de transactions.

8. Réponse A — Si la transaction concurrente fait un ROLLBACK, la valeur lue n'a jamais existé : READ COMMITTED suffit à éviter ce phénomène.

9. Réponse B — Plusieurs lecteurs peuvent partager un verrou de lecture ; l'écriture exige un verrou exclusif, incompatible avec tous les autres.

10. Réponse B — GRANT SELECT, INSERT ON boutique.commande TO 'caissier'... : ni opération superflue, ni table superflue — chaque droit inutile est un risque en cas d'erreur ou de fuite du mot de passe.

Résumé — les notions importantes

• Une table qui mélange plusieurs sujets crée redondance et anomalies d'insertion, de modification et de suppression.

• Les dépendances fonctionnelles traduisent les règles du métier ; la clé est le plus petit ensemble d'attributs qui détermine tous les autres.

• Normaliser = décomposer sans perte : 1FN (valeurs atomiques), 2FN (aucune dépendance partielle), 3FN (aucune dépendance transitive), BCNF (tout déterminant est une super-clé).

• Une transaction est validée par COMMIT ou annulée par ROLLBACK ; les propriétés ACID (Atomicité, Cohérence, Isolation, Durabilité) en font un mécanisme fiable, appuyé sur le journal.

• La concurrence mal contrôlée produit perte de mise à jour, lectures impropres ou non reproductibles et lignes fantômes ; le niveau d'isolation et les verrous (partagés/exclusifs) y répondent, au prix de possibles interblocages.

• Droits d'accès : principe du moindre privilège avec CREATE USER, GRANT, REVOKE et les rôles ; SQLite n'a pas de comptes et s'appuie sur les droits du fichier.

• Déclencheurs pour l'audit et les garde-fous automatiques, procédures stockées pour les opérations métier : deux mécanismes serveur qui protègent la base quel que soit le programme qui l'utilise.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Base de données #12: Cours intensif en conception de base de données

Chaîne : coursaline - Rachid Saad · Langue : Français · Niveau : Intermédiaire

Un cours intensif de conception qui fait la part belle aux dépendances fonctionnelles pour construire le modèle conceptuel. À regarder pour consolider la leçon 1 avant d'attaquer les décompositions.

https://www.youtube.com/watch?v=3w27DwqYMKk

Leçon 1 — Coursaline.com: Exercice 2 sur la modélisation des données (MCD)

Chaîne : coursaline - Rachid Saad · Langue : Français · Niveau : Intermédiaire

Un exercice complet de modélisation corrigé pas à pas, où les dépendances fonctionnelles servent à justifier chaque choix du modèle. Parfait pour s'entraîner à repérer les DF dans un énoncé métier.

https://www.youtube.com/watch?v=_nV4hno9NqE

Leçon 2 — Normalisation des Bases de Données Relationnelles : Deuxième Forme Normale (2FN)

Chaîne : KLN Technology · Langue : Français · Niveau : Intermédiaire

La vidéo se concentre sur la deuxième forme normale et le traitement des dépendances partielles, dans une série consacrée à la normalisation. Elle complète directement les décompositions de la leçon 2.

https://www.youtube.com/watch?v=nzD-cXS3bcM

Leçon 2 — □ 100 QCM pour Réviser les Bases de Données Relationnelles ! □

Chaîne : WayToLearnX · Langue : Français · Niveau : Intermédiaire

Cent questions de révision corrigées sur le modèle relationnel, les clés, la redondance, les dépendances fonctionnelles et les formes normales 1FN à 3FN. Idéale en auto-évaluation après les leçons 1 et 2 et avant les TD.

https://www.youtube.com/watch?v=MrKZIl2ttgc

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Gardarin, G. — « Bases de données » (Eyrolles) : ouvrage francophone de référence sur le modèle relationnel, la normalisation et les transactions.
- Date, C. J. — « An Introduction to Database Systems » (Addison-Wesley) : la référence classique sur les dépendances fonctionnelles et les formes normales.

• Documentations officielles : MySQL (dev.mysql.com/doc — transactions InnoDB, GRANT/REVOKE, déclencheurs) et SQLite (sqlite.org/lang.html — CREATE TRIGGER, BEGIN/COMMIT/ROLLBACK, SAVEPOINT) ; tous les exemples exécutables du cours ont été testés sous SQLite, les exemples propres à MySQL sont signalés.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !